

ARSENA Claire - TD4

Introduction à l'intelligence artificielle

▼ Exercice 1

Question 1

```
homme(albert). homme(jean). homme(paul). homme(bertrand).  
homme(louis). homme(benoit).  
  
femme(marie). femme(germaine). femme(sophie). femme(christiane).  
femme(simone).  
  
parent(louis, benoit). parent(albert, jean). parent(jean, simone).  
parent(jean, paul).  
parent(paul, sophie). parent(paul, bertrand). parent(marie, sophie).  
parent(marie, bertrand).  
parent(germaine, jean). parent(christiane, simone). parent(christiane, paul).  
parent(simone, benoit).
```

Question 2

```
pere(X, Y) :- homme(X), parent(X, Y).  
  
mere(X, Y) :- femme(X), parent(X, Y).  
  
grand_parent_de(X, Z) :- parent(X, Y), parent(Y, Z).  
  
avoir_un_enfant(X) :- parent(X, _).  
  
etre_un_enfant(X) :- parent(_, X).  
  
oncle(X, Y) :- homme(X), parent(P, Y), frere_ou_soeur(X,
```

P).

```
tante(X, Y) :- femme(X), parent(P, Y), frere_ou_soeur(X, P).
```

```
cousin(X, Y) :- homme(X), grand_parent_de(G, X), grand_p  
arent_de(G, Y), X \= Y.
```

```
cousine(X, Y) :- femme(X), grand_parent_de(G, X), grand_  
parent_de(G, Y), X \= Y.
```

```
ancetre(X, Y) :- parent(X, Y).
```

```
marie(X, Y) :- pere(X, Z), mere(Y, Z).
```

```
marie(X, Y) :- mere(X, Z), pere(Y, Z).
```

```
8 ?- tante(X, Y).  
X = simone,  
Y = sophie .
```

simone est la tante de sophie

```
9 ?- cousine(X, Y) .  
X = sophie,  
Y = benoit .
```

sophie est la cousine de benoit

```
10 ?- cousin(X, Y).  
X = bertrand,  
Y = benoit .
```

bertrand est le cousin de benoit

```
11 ?- ancetre(X, Y).  
X = louis,  
Y = benoit .
```

louis est l'ancetre de benoit

▼ Exercice 2

```
rond(sphere).
```

```
nonrond(X) :- rond(X), !, fail.
```

```
nonrond(_).
```

```
neg(P) :- P, !, fail.
```

```
neg(_).
```

```
15 ?- nonrond(sphere).  
false.  
  
16 ?- neg(rond(sphere)).  
false.  
  
17 ?- nonrond(terre).  
true.  
  
18 ?- neg(rond(terre)).  
true.
```

voici les vérifications effectuées dans le terminal

▼ Exercice 3

```
element_de(X, [X|_]).
```

```
element_de(X, [_|Q]) :- element_de(X, Q).
```

```
elements_impairs_de([], []).
```

```
elements_impairs_de([X], [X]).
```

```
elements_impairs_de([X, _|Q], [X|LI]) :- elements_impairs_de(Q, LI).
```

```
inverse([], []).
```

```
inverse([T|Q], LInv) :- inverse(Q, QInv), append(QInv, [T], LInv).
```

```
21 ?- elements_impairs_de([0, 1, 2, 3, 4, 5], LI).  
LI = [0, 2, 4].
```

```
22 ?- inverse([a, b, c], L).  
L = [c, b, a].
```

▼ Exercice 4

```

hors_de(_, []).
hors_de(X, [T|Q]) :-
    dif(X, T),
    hors_de(X, Q).

```

```

30 ?- hors_de(4, [1, 2, 3]).
true.

31 ?- hors_de(2, [1, 2, 3]).
false.

```

▼ Exercice 5

```

differeents([]).
differeents([T|Q]) :-
    hors_de(T, Q),
    differeents(Q).

```

```

33 ?- differeents([1, 2, 3, 4]).
true.

34 ?- differeents([1, 2, 3, 1]).
false.

```

▼ Exercice 6

```

mouvement(0, _, _, _, []).
mouvement(N, Start, Inter, End, S) :-
    N > 0,
    N1 is N - 1,
    mouvement(N1, Start, End, Inter, S1),
    append(S1, [Start -> End], S2),
    mouvement(N1, Inter, Start, End, S3),
    append(S2, S3, S).

```

```

2 ?- mouvement(3, a, b, c, S).
S = [(a->c), (a->b), (c->b), (a->c), (b->a), (b->c), (a->c)] .

```